Name	**Abdullah Sajid**
Reg no	**B23F0001AI062**
Section	**Ai Blue**
Subject	**CN**
TEACHER	**Adnan Iqbal**

ASSIGNMENT 1

1) What is the name of the website?

Observed hostname (SNI): chrome.cloudflare-dns.com

Evidence: the SNI string chrome.cloudflare-dns.com appears in the QUIC TLS ClientHello inside the capture (file byte area shown below).
Excerpt around the SNI (ASCII inside the pcap at file byte offset ≈ 0x26c8 / decimal 9928–9940):

... 00 00 1e 00 1c 00 00 19 63 68 72 6f 6d 65 2e 63 6c 6f 75 64 66 6c 61 72 65 2d 64 6e 73 2e 63 6f 6d 00 ...

^^^

chrome.cloudflare-dns.com

---

2) Which packet contains the Initial QUIC handshake and what information is exchanged there?

Where (approx): the QUIC *Initial* long-header packet that carries the TLS ClientHello is present near file byte offset 0x2668 (≈ 9832 decimal) in the pcap you uploaded.

What is exchanged in the Initial QUIC handshake (observed in that packet):

- QUIC long header (Initial) from the client — contains client connection ID and the QUIC *version* field.
- TLS 1.3 ClientHello embedded in a QUIC CRYPTO frame. The ClientHello includes:
    - SNI (server name): chrome.cloudflare-dns.com
    - Cipher suites (TLS 1.3 ciphers visible e.g. 0x1301, 0x1302, 0x1303 in the ClientHello).
    - Extensions including ALPN (shows h3 / h2 / http/1.1 variants in the ALPN extension list), supported groups, signature schemes, etc.
    - Other TLS ClientHello fields (client random, session id, extensions).

So the Initial QUIC packet carries transport-level QUIC info (connection IDs, packet number space, version) plus the *TLS ClientHello inside CRYPTO* that advertises SNI, ALPN and ciphers.

(If you open this trace in Wireshark: filter udp.port==443 and look for the first packet of type Initial — expand the QUIC → CRYPTO frame → TLS → ClientHello to see the SNI and ALPN.)

---

3) Which QUIC packet contains the TLS ClientHello?

The TLS ClientHello is inside the CRYPTO frame of the client's QUIC Initial (long header) packet. In the raw file it appears around file byte offset 0x2668 (≈ 9832); the ASCII SNI and TLS cipher-suite list are right there. In Wireshark you'll see the ClientHello under: QUIC → CRYPTO → TLS → ClientHello for that Initial packet.

Hex snippet near that ClientHello (shows TLS cipher suites & SNI region):

... 13 01 13 02 13 03 c0 2b c0 2f c0 2c c0 30 ... 00 0e 00 0c ... 68 74 74 70 2f 31 2e 31 ... 63 68 72 6f 6d 65 2e 63 6c 6f 75 64 66 6c 61 72 65 2d 64 6e 73 2e 63 6f 6d ...

(13 01 13 02 13 03 are TLS1.3 ciphers; the ascii shows http/1.1/ALPN and then the chrome.cloudflare-dns.com SNI.)

---

4) Which QUIC version is used in your trace?

I found a QUIC version field value of 0x0000001e (hex) appearing in the Initial packet area of the capture (file offset near where the ClientHello appears).

- That value appears as the 4-byte version field in the QUIC long header in this trace.

(Interpretation: the version field in the Initial long header is 0x0000001e — I report the literal value observed in the pcap. If you prefer, you can confirm in Wireshark by enabling the QUIC protocol dissector and looking at the Initial packet → QUIC header → Version field.)

---

5) Where are 0-RTT or 1-RTT keys first used?

- 0-RTT (early data) would be present if the client sends a short-header packet carrying 0-RTT data before the handshake completes. In this capture I did not find a clear ASCII 0-RTT payload (0-RTT use is signaled by a short-header encrypted packet sent by the client immediately after sending the ClientHello with a PSK; that appears as a protected short-header packet).

- I *did* find the first short-header (protected) QUIC packets — which are the ones that use 1-RTT keys — shortly after the handshake packets. The first candidate short-header protected packet occurs at an approximate file offset 0xE3B9 (≈ 58257 decimal) in your pcap (this is the first region, after the ClientHello/handshake frames, where the payload looks like protected (non-printable) data carried in short-header packets). That indicates 1-RTT keys were in use from that point onward for application traffic.

In Wireshark you can confirm by looking for the first packet shown as Short Header, and Wireshark will decode it as Protected (1-RTT) once the handshake completes and keys are derived.

---

6) First packet carrying application data (HTTP/3) — how it differs vs HTTP over TCP

First application-data packet (HTTP/3):
The first application data (HTTP/3) frames appear after the 1-RTT keys are in use — i.e., in the short-header protected packets (the same region around file offset ≈ 58257 onward). Because HTTP/3 application data is encrypted inside QUIC 1-RTT packets, the actual HTTP/3 payload bytes are protected and mostly binary (QPACK-encoded headers, HTTP/3 frames like HEADERS/DATA inside QUIC STREAM frames).

How HTTP/3 over QUIC differs from HTTP over TCP:

- Transport layer: HTTP/3 runs over QUIC (UDP); HTTP/1.1/2 run over TCP.
- Connection and multiplexing: QUIC provides stream multiplexing without head-of-line blocking at the transport layer. In TCP, packet loss blocks all streams on the same TCP connection until the lost packet is retransmitted (head-of-line blocking for HTTP/2 multiplexed streams). QUIC isolates streams so one stream's loss doesn't block others.
- Encryption & handshake: QUIC integrates the TLS 1.3 handshake into transport (TLS handshake occurs inside QUIC CRYPTO frames). The TLS handshake is not on top of TCP; QUIC performs transport + crypto together and can enable 0-RTT/1-RTT faster resumptions.
- Connection identifiers & mobility: QUIC uses connection IDs so the connection survives client IP changes (TCP connections would break).
- Header compression: HTTP/3 uses QPACK (a variant of HPACK adapted to avoid head-of-line blocking) rather than the HPACK used by HTTP/2 over TCP.

- Visibility: HTTP payload and headers are inside encrypted QUIC frames (so you won't see GET/Host in plaintext in the capture once 1-RTT keys are used), unlike some unencrypted HTTP/1.1 (or if you had TLS over TCP you'd still see no plaintext).